

AI-Powered Support Ticket Analytics System

DOTMappers AI Engineer Assessment

An end-to-end AI system for querying support ticket data using natural language, detecting anomalies, exposing REST APIs, and providing a minimal user interface.

Overview

This project was developed as part of the **DOTMappers AI Engineer Assessment**.

The goal is to build an AI-powered analytics system that:

- Ingests support ticket CSV data.
- Answers natural language questions using an LLM.
- Detects anomalies in support operations.
- Exposes functionality through REST APIs.
- Provides a simple UI for interaction.

The design emphasizes:

- Simplicity
 - Reliability
 - Zero-cost execution
 - Production-style architecture
 - Easy scalability
-

Why This Architecture?

The dataset contains only **500 records**, making heavy infrastructure unnecessary.

Instead of introducing databases, vector stores, or multi-agent systems, the solution uses:

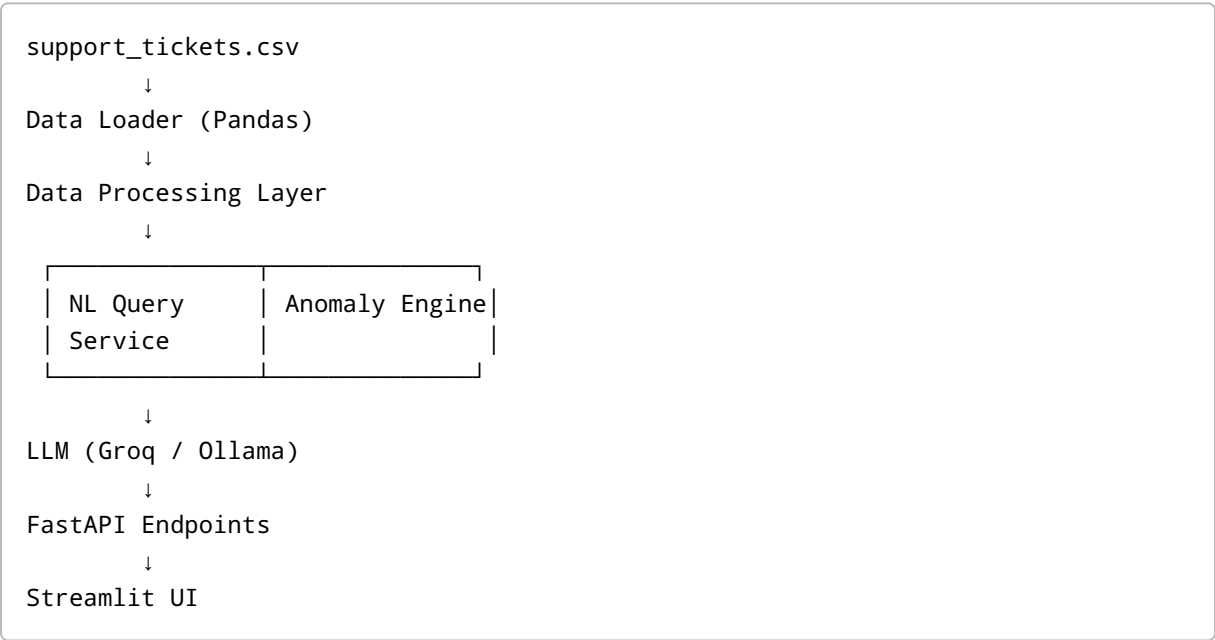
- Pandas for analytics
- FastAPI for APIs
- Streamlit for UI
- Groq/Ollama for LLM capabilities

Design Philosophy

LLM = Understand the question
Python = Perform calculations

This separation prevents hallucinations and ensures deterministic answers.

System Architecture



Technology Stack

Component	Technology
Language	Python
Data Processing	Pandas
API Framework	FastAPI
UI	Streamlit
LLM	Groq Llama3 / Ollama
Validation	Pydantic

Component	Technology
Containerization	Docker
Anomaly Detection	Rule-Based + Optional Isolation Forest

Project Structure

```
project/
|
|— data/
|   support_tickets.csv
|
|— app/
|   |
|   |— main.py
|   |— routes/
|   |   query.py
|   |   anomaly.py
|   |   health.py
|   |
|   |— services/
|   |   llm_service.py
|   |   query_engine.py
|   |   anomaly_engine.py
|   |
|   |— utils/
|   |   loader.py
|   |
|   |— models/
|   |   schemas.py
|
|— ui/
|   streamlit_app.py
|
|— requirements.txt
|— README.md
|— docker-compose.yml
```

Data Flow

```
User Question
  ↓
LLM understands intent
  ↓
Structured operation generated
  ↓
Pandas performs analytics
  ↓
Response returned to user
```

Example:

User Query

```
Which agent has the lowest average customer rating?
```

LLM Output

```
{
  "metric": "avg_customer_rating",
  "group_by": "agent_id",
  "sort": "ascending"
}
```

Python Executes

Pandas computes the actual answer and returns it to the user.

REST API Endpoints

Health Check

```
GET /health
```

Response:

```
{
  "status": "running"
}
```

Natural Language Query

```
POST /query
```

Request:

```
{
  "question": "How many open tickets exist?"
}
```

Response:

```
{
  "answer": "There are 45 open tickets."
}
```

Anomaly Detection

```
GET /anomalies
```

Response:

```
[
  {
    "ticket_id": "TKT-204",
    "reason": "Critical ticket unresolved for more than 24 hours"
  }
]
```

Summary Endpoint (Optional)

```
GET /summary
```

Returns:

- Total tickets
 - Open tickets
 - Resolved tickets
 - Average customer rating
-

Streamlit UI

The UI contains three sections:

1. Ask Questions

Users can enter natural language queries and receive AI-generated answers.

2. Anomaly Dashboard

Displays flagged tickets along with anomaly reasons.

3. Dataset Statistics

Shows:

- Total tickets
 - Open tickets
 - Resolved tickets
 - Average rating
-

Anomaly Detection Strategy

Rule-Based Anomalies

Critical Tickets Older Than 24 Hours

```
(priority == "Critical")
&
(status != "Resolved")
&
(age_hours > 24)
```

Abnormally Long Resolution Time

Threshold:

```
mean + (2  standard deviation)
```

Tickets exceeding the threshold are flagged.

Optional Machine Learning Enhancement

Isolation Forest can be used on:

- response_time_hrs
- resolution_time_hrs
- customer_rating

This is optional because rule-based logic already satisfies the requirements.

Why This Approach?

Advantages

- ✓ Simple
- ✓ Fast
- ✓ Zero-cost

- ✓ Deterministic outputs
 - ✓ Easy to explain during walkthrough
 - ✓ Minimal infrastructure
 - ✓ Production-friendly structure
-

Alternative Architectures Considered

1. RAG Architecture

```
CSV
↓
Chunking
↓
Embeddings
↓
Vector Database
↓
Retriever
↓
LLM
```

Why Not?

RAG is suitable for unstructured documents, not tabular analytics.

Questions like:

- "How many tickets are open?"
- "Average customer rating?"

require computations rather than semantic retrieval.

2. SQL + LangChain Agent

```
CSV
↓
SQLite
```

↓
SQL Agent
↓
LLM

Why Not?

Pros:

- Scalable

Cons:

- Extra complexity
- Possible SQL hallucinations
- Unnecessary for 500 rows

3. Vector Database (FAISS / Chroma)

Why Not?

Vector databases are designed for semantic search, not:

- Aggregations
- Counts
- Statistical analysis

4. Multi-Agent Systems

Supervisor Agent
↓
Query Agent
Analytics Agent
Anomaly Agent

Why Not?

Overengineering for a small dataset.

Adds complexity without providing significant benefits.

LLM Choice

Preferred Option: Groq + Llama 3

Advantages:

- Free
 - Fast inference
 - No local GPU required
-

Alternative Option: Ollama

Advantages:

- Fully offline

Disadvantages:

- Requires model installation
 - Slightly harder setup
-

Scalability Roadmap

Phase 1 (Current)

CSV + Pandas

Suitable for:

- Hundreds of records
-

Phase 2

PostgreSQL

Suitable for:

- Hundreds of thousands of records
-

Phase 3

SQL Agent + Caching

Suitable for:

- Millions of records
-

Phase 4

Microservices + Kafka + Redis

Suitable for enterprise-scale deployments.

Setup Instructions

Clone Repository

```
git clone <repository-url>
cd project
```

Create Environment

```
python -m venv venv
```

Activate:

```
source venv/bin/activate
```

or

```
venv\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Run FastAPI

```
uvicorn app.main:app --reload
```

Run Streamlit

```
streamlit run ui/streamlit_app.py
```

Future Improvements

- Add authentication.
 - Add caching.
 - Add database support.
 - Add ML-based anomaly detection.
 - Add conversation memory.
 - Add dashboard visualizations.
 - Deploy with Docker.
-

Conclusion

This solution prioritizes:

1. Correctness
2. Simplicity
3. Explainability
4. Maintainability
5. Scalability

The architecture intentionally avoids unnecessary complexity and uses the LLM only for understanding natural language while delegating all computations to deterministic Python logic.